Community Scam Map: Student Brief

Oct 29, 2026 · @Prasanth

Build an app where people warn each other about scam phone numbers, UPI IDs and links, and check one before they trust it. This brief tells you what to build, what we give you and how you are scored. Read it once end to end before you write any code.

The problem

Ravi's father gets a video call from a man in a police uniform. The man says a parcel with drugs was found in his name and he must pay to avoid arrest. He stays on the call for three hours and loses ₹2 lakh. This is called a "digital arrest" scam, and it is fake.

Ravi tells his family. Nobody else hears about it. The next day, the same number calls Priya across the city. She has no way to check whether anyone has already reported it.

Scams like this reach people every day through calls, SMS, WhatsApp, QR codes and UPI payment requests. What is missing is a simple, trusted place to ask: **"Has anyone reported this number, UPI ID or link as a scam?"**

That is what you will build.

What you will build

An Android app or mobile website, plus a server behind it. It needs five features. The first four are required; the fifth earns bonus points.

#	Feature	Example	Must work like this
1	**Report a scam**	Ravi taps "Phone call", "Just now", pastes the number and posts	Takes under 60 seconds. Saves the area, like "HSR Layout", never the exact address.
2	**Look it up**	Priya pastes a number and sees "Reported by 7 people near you"	Answers in under half a second with one of four results: Known scam, Suspicious, No reports, Verified business.
3	**Scam map**	Shows that HSR Layout has 12 scam reports this week	Shows only the last 7 days, grouped by area.
4	**Alerts**	"Scam reports near you doubled today"	Fires only when something is unusual, never for every report.

#	Feature	Example	Must work like this
5	**Dispute (bonus)**	A bakery says "that is our real number"	The number is marked as under review.

The same number can be typed many ways: `+91 98765 43210`, `09876543210`, `98765-43210`. Your app must treat all of these as one number.

The tricky part: fake reports

Building the five features is the easy half. The hard half is that people will try to misuse your app. If you simply count reports, these tricks will work:

- **Report-bombing.** A rival shop asks 40 friends to report a real bakery's number as a scam.
- **Fake accounts.** One person makes 22 accounts from the same Wi-Fi and reports one number.
- **Spamming.** One person reports the same number six times in an hour.
- **Junk.** Someone types "12345" or "test" as the number.

If any of these work, an innocent bakery gets labelled a scam. So you need **trust rules**: rules that decide how much each report should count. Some ideas:

- A new user's report counts less than one from a user whose earlier reports turned out to be right.
- Many reports from the same internet connection within a short time count as one person.
- One person reporting many times is still one person.
- A report with a personal description counts more than a copy-pasted one.
- Junk numbers are rejected straight away.

Be careful: a real scam campaign can also cause a sudden wave of reports. The difference is that real reports come from **many different, unrelated people** writing in their own words. Your rules must stop the fakes without missing the real scams.

On judging day, we load test reports with all of these tricks into your server. Make your own fake reports while you build, to check that your rules stop them.

Privacy rules

People will only report scams if they feel safe doing it. **Breaking any of these rules disqualifies your team, however good the rest is.**

1. **Keep reporters anonymous.** Never store or show a reporter's name, phone number or exact location. Save the area only.
2. **Hide numbers until they are confirmed.** Show `98XXXXX210` until at least 3 different people have reported it. One angry person should not be able to label someone publicly.
3. **Say "reported", not "scammer".** Write "Reported by 7 people near you", never "This person is a fraud".
4. **Don't dig up personal data.** No collecting names or details from social media or people-search websites.
5. **No personal data in logs.**

Ship it: a server and a licence

Imagine a bank wants to buy your app and run it on its own server. You want it to stop working if the bank stops paying, like a prepaid mobile plan that stops when the recharge runs out. This part tests whether you can build that.

Step 1: put it on a real server. Run your server on a cloud machine with a public web address, such as `https://yourteam-scammap.onrender.com`, not on your laptop. Free options like Render, Railway, Fly.io or a free cloud VM are fine. Use HTTPS.

Step 2: add a licence file. A licence is a small file that says who may use the app and until when:

```
{
  "licensee": "Demo Bank",
  "expires_at": "2026-11-15T10:05:00+05:30",
  "signature": "MEUCIQDx7k..."
}
```

The signature stops people editing the date. You create it with a **private key** that stays on your laptop. Your server checks it with the matching **public key**, which is built into your code. If anyone changes the date, the check fails. Libraries like PyNaCl in Python or `jsonwebtoken` in Node.js make this a few lines.

Step 3: check it on every request. Your server checks the licence regularly, for example on every request or every few minutes:

- **Valid:** everything works.
- **Expired:** every request except `/health` and `/license` gets the reply `403 {"error": "LICENSE_EXPIRED"}`. Your app shows "Licence expired. Contact your provider." Do not delete any data.

- **Renewed:** when a new licence file is uploaded through `POST /license`, everything works again **without restarting the server**.

What judges will ask you to show, live:

1. The app working, and `GET /license` showing who it is licensed to and when it expires.
2. You upload a licence that expires in 5 minutes.
3. After 5 minutes, a lookup fails and the app shows the expired screen.
4. You upload a new licence, and the app works again within a minute, with old data still there.

Bonus thinking: what if someone sets the server clock back a year to cheat? A simple fix is to save the latest time your server has seen and refuse if the clock ever goes backwards.

What we give you

You don't need any access to Mitigata systems. Everything your server must follow is in the **Appendix** at the end:

- **The API rules:** the web addresses your server must have, what each one receives and what it must reply. Our test tool uses these on judging day.
- **The area codes:** the 25 areas the test tool uses, with their map points.
- **Verified numbers and scam types:** the numbers that must show "Verified business", and the names of the scam types and report channels.

There is no practice data and no test tool to download. Write your own test reports, including fake ones like the tricks above, and run them against your server as you build. Public helplines such as 1930 and cybercrime.gov.in can be used directly in your app.

How you are scored

You are scored out of 100, plus up to 7 bonus points. The first four rows are scored by a test tool the judges run against your live server, using the API rules in the appendix. The last three are scored by a judge watching you.

#	What we check	Points	How we check it	Full marks when	Zero when
1	**Stops fake reports**	20	The test tool loads reports full of tricks: 40 friends reporting a real bakery, 22 fake accounts on one Wi-Fi, one person reporting six times, junk like "12345". It checks none of these makes a number "Known scam".	Every trick is stopped	One person reporting many times can make a number "Known scam"
2	**Lookup gives the right answer**	15	The test tool checks real scams come back "Known scam", unreported numbers come back "No reports", `+91 98765 43210` and `09876543210` give the same answer, and answers take under half a second.	Right answers, every format matches, fast	Most answers are wrong
3	**Map and alerts**	10	The test tool checks an area with a sudden jump gets an alert, a normal area does not, a new kind of scam gets an alert, and month-old reports are not on the map.	Both work	Neither works
4	**Privacy**	15	The test tool checks numbers with fewer than 3 reporters are hidden and the map never shows a person's location. A judge reads your result screen: "Reported by 7 people near you" is good; "This person is a fraud" loses up to 5 points.	Hidden numbers, anonymous reporters, careful wording	A reporter can be identified: you are disqualified
5	**Server and licence**	15	You show the four-step licence demo live on your server.	All four steps work on an https address with no restart	It only runs on a laptop, or nothing stops it

#	What we check	Points	How we check it	Full marks when	Zero when
6	**Easy to report**	15	A judge reports a scam on a phone with a stopwatch, then tries it in Hindi or using only the icons.	Under 60 seconds, and usable in Hindi or from icons	Over 2 minutes, or a login before the form
7	**Demo and code**	10	A judge tries to run your project from the README, reads how you explain your trust rules and licence, and listens to your pitch.	Easy to run, clear README, clear pitch	It cannot be run

Between full marks and zero, judges give part marks. For example, a licence demo that works but needs a server restart gets about half.

Bonus, up to 7: 3 points if verified business and helpline numbers show "Verified business", 2 for the dispute feature, and 2 if fake reports never set off an alert.

Tie-break: more bonus points first, then the faster report time.

Important: judges use different data. On judging day we load a new, secret set of reports into your server. You won't see it in advance. Rules written for one specific number will not work, so if you write code like `if number == "+919876543210": return "SUSPICIOUS"`, it will fail, and we check for it. Build real rules.

Automatic disqualification if a reporter can be identified, exact locations are stored, personal data is scraped, or the code was written before the event.

A suggested 24-hour plan

This is only a suggestion. The one thing we strongly recommend is putting something on a real server in the first two hours.

Stages	What to do
Stage 1	Read this brief, including the appendix. Deploy a "hello world" to your cloud server.
Stage 2	Build report and lookup with a real database. Make all number formats match.
Stage 3	Build your trust rules. Test them with your own fake reports after every change.

Stages	What to do
Stage 4	Map, alerts and number masking.
Stage 5	Licence file, licence check and the "Licence expired" screen.
Stage 6	Put everything on your server. Practise the licence demo three times.
Stage 7	Hindi or icon-based report screen, README and pitch.

What to submit

- [ ] Your code in a Git repository
- [ ] Your server's public web address, working during judging
- [ ] Your app: an APK file, a mobile website link, or both
- [ ] A README explaining how to run it, how your trust rules work, and how your licence works
- [ ] A demo video of up to 3 minutes
- [ ] Two licence files ready for the demo: one that expires in 5 minutes and one long one

Tips

- Keep a small set of your own test reports, real-looking and fake, and run it after every change.
- Explain your trust rules in the README. Judges will ask why a fake-reported number was not labelled a scam.
- Keep the report screen short. Nobody fills a long form after a scam call.
- Never put your private licence key in your Git repository.

Common mistakes

- Leaving the server setup to the last hour.
- Showing full phone numbers that only one person has reported.
- Checking the licence only when the server starts, so it never expires during the demo.
- Deleting data when the licence expires. Lock it, don't delete it.

Questions

- **Can we use AI tools or ChatGPT-style APIs?** Yes, for example to understand free-text reports. Your trust rules should still be your own, and you must be able to explain them.

- **App or website?** Either works. A mobile website must work well on a phone.
- **Any language or framework?** Yes. Just follow the API rules in the appendix so our test tool can talk to your server.
- **Do we need Mitigata's code?** No. Everything you need is in this brief.
- **Where do we ask questions?** In the hackathon channel.

Appendix

A. API rules

On judging day our test tool talks to your server at these seven addresses. Use them exactly as written, with JSON in and out. Extra fields in your replies are fine.

#	Address	What it does
1	`GET /health`	Says the server is alive
2	`POST /reports`	Receives one scam report
3	`GET /lookup`	Checks a number, UPI ID, link or SMS sender
4	`GET /map`	Lists scam reports by area for the last 7 days
5	`GET /alerts`	Lists alerts for an area
6	`POST /admin/reset`	Deletes all reports, so the judges start clean
7	`POST /challenges`	A business disputes a number (bonus)

Your server also needs `GET /license` and `POST /license`, described in "Ship it".

Six rules the test tool relies on

1. **Time comes from the report.** Times look like `2026-10-27T18:21:04+05:30`. Work out "last 7 days" and "last 24 hours" from each report's `occurred_at`, compared with your server's clock.
2. **The reporter comes from the report.** The test tool runs on one laptop but pretends to be hundreds of people. Treat `reporter_id` and `ip_prefix` in the report as the reporter's identity, not the laptop's real address.
3. **One identifier, many ways of typing it.** Store each one in a single standard form:
   - Phone: `+91` and 10 digits. `+91 98765 43210`, `09876543210`, `98765-43210` and `919876543210` all become `+919876543210`.
   - UPI ID: lower case, no spaces. `Ravi12@OKAXIS` becomes `ravi12@okaxis`.

- Link: `https://`, no `www.`, nothing after `?`, no `/` at the end. `http://www.sbi-kyc.in/pay?utm=sms` becomes `https://sbi-kyc.in/pay`.
- SMS sender: the 6-letter code, without the 2-letter prefix the phone adds. `VM-HDFCBK`, `AD-HDFCBK` and `HDFCBK` are all `HDFCBK`.

4. **Slowing someone down is fine.** If you rate-limit a reporter, reply `429`. It is not counted against you.
5. **No login on these seven addresses.** The test tool sends no password or key.
6. **Reset must work** on the server you show the judges.

1. GET /health

Reply `200` with `{"ok": true}`.

2. POST /reports

The test tool sends:

```
{
  "report_id": "rep_000123",
  "occurred_at": "2026-10-27T18:21:04+05:30",
  "reported_at": "2026-10-27T18:40:04+05:30",
  "reporter_id": "r_7c2d91aa",
  "ip_prefix": "103.21.58",
  "channel": "Phone call",
  "scam_type": "digital-arrest",
  "identifier_type": "phone",
  "identifier_value": "+91 98765 43210",
  "pincode": "560102",
  "locality": "HSR Layout",
  "city": "Bengaluru",
  "note": "Video call from a man in police uniform...",
  "has_screenshot": false,
  "amount_lost_inr": 0
}
```

`identifier_type` is `phone`, `upi`, `url` or `sender_id`. `channel` and `scam_type` come from Appendix C. The note may be in English, Hindi or Hinglish, or empty.

Reply:

- `201` with `{"report_id": "...", "case_id": "...", "merged": true}`. Set `merged` to `true` when the report joined an existing report about the same identifier.
- `400` with `{"error": "..."}` when the identifier, area code or time is not valid. Some test reports are junk on purpose; rejecting them is correct.

- `429` when you are slowing this reporter down.

3. GET /lookup

Example: `GET /lookup?type=phone&value=%2B91%2098765%2043210` (the value is URL-encoded).

```
{
  "verdict": "KNOWN_SCAM",
  "report_count": 7,
  "distinct_reporters": 7,
  "first_seen": "2026-10-14T09:12:00+05:30",
  "last_seen": "2026-10-27T18:21:04+05:30",
  "scam_type": "digital-arrest",
  "display_value": "+919876543210",
  "evidence": [
    {"occurred_at": "2026-10-27T18:21:04+05:30", "channel": "Phone call",
"scam_type": "digital-arrest", "pincode": "560102"}
  ],
  "challenged": false
}
```

- `verdict` is exactly one of `KNOWN_SCAM`, `SUSPICIOUS`, `NO_REPORTS`, `VERIFIED_BUSINESS`.
- `distinct_reporters` counts **different people** after your trust rules. One person reporting six times is 1.
- `display_value` is what your app would show. Hide it until at least 3 different people have reported it: `98XXXXX210` for a phone, `r***@okaxis` for a UPI ID, `sbi***.in/...` for a link, `HD****` for a sender.
- `evidence` never includes who reported, their IP or their note.
- A number nobody reported is `200` with `NO_REPORTS`, not `404`. A value that cannot be valid, like `12345`, is `400`.

4. GET /map

Example: `GET /map?pincode=560102&days=7&radius_km=2`. If `days` is missing, use 7. If `radius_km` is missing, use 2.

```
{
  "clusters": [
    {"pincode": "560102", "locality": "HSR Layout", "city": "Bengaluru",
"count": 12,
     "top_scam_type": "digital-arrest", "severity": "CRIT", "lat": 12.9116,
"lng": 77.6473}
  ]
}
```

One cluster per area within the radius that has reports in the window. Use the area's map point from Appendix B, never a reporter's location. `severity` is `CRIT`, `HIGH` or `MED`, on your own scale.

5. GET /alerts

Example: `GET /alerts?pincode=560102`

```
{"alerts": [
  {"type": "SPIKE", "pincode": "560102", "window": "24h", "count": 10,
"baseline_per_day": 2.0},
  {"type": "NEW_TYPE", "pincode": "560102", "scam_type": "job-offer",
"window": "24h"}
]}
```

- `SPIKE`: reports in the last 24 hours are at least double the daily average of the 7 days before, and at least 5.
- `NEW_TYPE`: a scam type reported in the last 24 hours that was not seen in that area in the 30 days before.
- No alerts means `{"alerts": []}`. Fake reports your trust rules ignored should not set off alerts.

6. POST /admin/reset

Delete every report and reply `200`.

7. POST /challenges (bonus)

```
{"identifier_type": "phone", "identifier_value": "+91 80 4100 2233",
 "claimant_name": "Fresh Bakes", "contact_email": "owner@example.com",
 "proof_note": "GST certificate and shop licence attached."}
```

Reply `201` with `{"challenge_id": "...", "status": "under_review"}`. After that, a lookup on that number shows `"challenged": true` and is never `KNOWN_SCAM` while under review.

B. Area codes

The test tool's reports use these 25 areas. Your server must accept them, and your map must place each area at the point given here, never at a reporter's own location. You may add more areas of your own.

Area code	Area	City	Latitude	Longitude
560102	HSR Layout	Bengaluru	12.9116	77.6473
560038	Indiranagar	Bengaluru	12.9784	77.6408
560034	Koramangala	Bengaluru	12.9352	77.6245
560066	Whitefield	Bengaluru	12.9698	77.7500
560076	BTM Layout	Bengaluru	12.9166	77.6101
560011	Jayanagar	Bengaluru	12.9299	77.5826
560037	Marathahalli	Bengaluru	12.9591	77.6974
560103	Bellandur	Bengaluru	12.9260	77.6762
560100	Electronic City	Bengaluru	12.8452	77.6602
560078	JP Nagar	Bengaluru	12.9077	77.5851
560003	Malleshwaram	Bengaluru	13.0035	77.5709
560010	Rajajinagar	Bengaluru	12.9915	77.5520
560024	Hebbal	Bengaluru	13.0358	77.5970
560064	Yelahanka	Bengaluru	13.1007	77.5963
560070	Banashankari	Bengaluru	12.9250	77.5468
400050	Bandra West	Mumbai	19.0596	72.8295
110016	Hauz Khas	Delhi	28.5494	77.2001
500081	Madhapur	Hyderabad	17.4486	78.3908
600020	Adyar	Chennai	13.0012	80.2565
411014	Viman Nagar	Pune	18.5679	73.9143
700091	Salt Lake Sector V	Kolkata	22.5790	88.4290
380015	Satellite	Ahmedabad	23.0260	72.5085
302017	Malviya Nagar	Jaipur	26.8570	75.8080

Area code	Area	City	Latitude	Longitude
226010	Gomti Nagar	Lucknow	26.8540	80.9990
682030	Kakkanad	Kochi	10.0159	76.3419

C. Verified numbers, scam types and channels

Verified numbers. A lookup on any of these must say `VERIFIED_BUSINESS`, never a scam. This earns 3 bonus points. The last three are made-up local shops for testing.

Type	Value	Belongs to
phone	1930	National Cybercrime Helpline
phone	1909	TRAI Do Not Disturb
phone	1800 1600	HDFC Bank customer care
phone	1800 2600	HDFC Bank customer care
phone	1800 1080	ICICI Bank customer care
phone	1800 1234	State Bank of India customer care
phone	1800 2100	State Bank of India customer care
phone	1860 419 5555	Axis Bank customer care
phone	1860 266 2666	Kotak Mahindra Bank customer care
sender_id	HDFCBK	HDFC Bank SMS sender
sender_id	ICICIB	ICICI Bank SMS sender
sender_id	SBIINB	State Bank of India SMS sender
sender_id	AXISBK	Axis Bank SMS sender
sender_id	KOTAKB	Kotak Mahindra Bank SMS sender
url	https://cybercrime.gov.in	National Cyber Crime Reporting Portal
url	https://sancharsaathi.gov.in	Sanchar Saathi
phone	+91 80 4100 2233	Fresh Bakes, Indiranagar (made up)
phone	+91 80 2553 0190	Koramangala Pharmacy (made up)
upi	freshbakes@okaxis	Fresh Bakes, Indiranagar (made up)

Note that `1800 1600` is a toll-free number, not a 10-digit mobile. Your phone rules must accept short helplines like `1930` and toll-free numbers starting `1800` or `1860`.

Scam types. The `scam_type` field in a report is one of these names. Use them for report options and to show the right safety advice.

scam_type	What it is
`digital-arrest`	Digital arrest video call: fake police say you are under arrest
`fake-kyc-link`	Fake KYC update link that copies a bank's page
`upi-refund-qr`	UPI refund QR scam: "scan to receive" actually sends money
`investment-group`	WhatsApp investment group with fake profits
`courier-customs`	Parcel held at customs, "pay a fine"
`job-offer`	Work-from-home task job that asks for deposits
`lottery-prize`	Lottery or prize win that needs a fee
`loan-app-harassment`	Instant loan app that threatens your contacts
`electricity-disconnection`	"Power cut tonight" SMS
`fake-customer-care`	Fake customer care number from a search result
`otp-call`	OTP or card verification call
`sextortion`	Video call extortion

Accept any other value too; treat it as "other" rather than rejecting the report.

Report channels. The `channel` field is one of: `Phone call`, `SMS or WhatsApp`, `QR code`, `Fake website`, `UPI request`, `Something else`.